

Demo 2: Paper Results Reproduction

From a published scientific paper to tested research software

Dr. Denis Anikiev (CIPR, KFUPM)

Guided by prompts, tests, and paper-grounded milestones

Demo Idea

What we start from

- a scientific paper in PDF
- a task brief and milestones
- no starter package code

What learners practice

- extracting an implementation spec from a paper
- asking an AI agent for small, testable steps
- validating equations before reproducing examples

Teaching point

The agent is most useful when the paper is turned into explicit interfaces, tests, and milestones.

Why This Is Different From Demo 1

Demo 1

- visual target is a scanned figure
- progress is judged by image comparison
- prompts refine geometry and viewpoint
- output is one reproducible figure generator

Demo 2

- target is a computational method
- progress is judged by tests and examples
- prompts refine equations, APIs, and validation
- output is a small research package scaffold

Input Package

Files that frame the work

- `input/TASK.md`: rebuild the core workflow
- `input/DATA.md`: paper reference and method context
- `input/OUTPUTS.md`: package deliverables
- `input/MILESTONES.md`: implementation sequence
- `prompts.md`: realistic agent prompts

Deliverables

- code in `src/`
- `requirements.txt` and `environment.yml`
- a `README.md` with assumptions and limitations

Paper Reference

Fang, X. and Chen, X. (2019). A fast and robust two-point ray tracing method in layered media with constant or linearly varying layer velocity. *Geophysical Prospecting*, 67(7).

<https://doi.org/10.1111/1365-2478.12799>

Geophysical Prospecting

EAGE
EUROPEAN
ASSOCIATION OF
GEOPHYSICISTS &
ENGINEERS

Original Article | [Full Access](#)

A fast and robust two-point ray tracing method in layered media with constant or linearly varying layer velocity

[Xinding Fang](#) [✉ Xiaofei Chen](#)

First published: 22 April 2019 | <https://doi.org/10.1111/1365-2478.12799> | [VIEW METRICS](#)

SFX Wissenschaftspark Albert Einstein [🔗](#)

📄 PDF 🗣️ CITE 🛠️ TOOLS ➦ SHARE

ABSTRACT

A fast and robust method for two-point ray tracing in one-dimensional layered media is presented. This method is applicable to layered models with constant or linearly varying isotropic layer velocity. For given model properties and source and receiver positions, a ray path can be uniquely determined once its ray parameter (i.e. horizontal slowness) is known. The ray parameter can be obtained by numerically solving the nonlinear offset (i.e. source–receiver horizontal distance) equation using Newton's method, which generally works well at near and mid offsets. However, Newton's method becomes hard to converge at large offsets due to the oversensitivity of offset to ray parameter. Based on the analysis of the characteristic of the offset equation, a modified ray parameter is proposed and used to replace the generic ray parameter in numerical calculation. Numerical experiments show that the iteration process becomes stable and converges rapidly with the modified ray parameter. Moreover, a rational function that asymptotically approximates the shape of the offset equation is introduced for obtaining good initial estimates of the modified ray parameter. Numerical tests show that this method is robust in any situation, and an accurate ray parameter can be obtained within two or three iterations for a wide range of model velocity structure and source–receiver distance. Furthermore, the proposed two-point ray tracing method is easy to implement.

Geophysical Prospecting

Volume 67, Issue 7
September 2019
Pages 1811–1824

Description

Contents of `DATA.md`

Scientific setting

- 1-D layered isotropic media
- source and receiver define a 2-D vertical plane
- ray parameter follows Snell's law
- large offsets make direct parameter solving difficult

Method hook

- The paper introduces a dimensionless q parameter
- The nonlinear offset equation is better behaved for Newton-style iteration.

Description of the Task

Contents of **TASK.md**

Build a small, inspectable Python package to implement the two-point ray tracing method from the paper.

First-pass behavior:

- define 1-D layered velocity models
- solve two-point ray tracing with the q-parameter
- compute ray parameter, ray path, and travel time
- trace direct and simple reflected rays
- plot 2-D ray paths over layered models
- reproduce selected paper-style examples

Target Package Shape

```
.  
|-- README.md  
|-- pyproject.toml  
|-- environment.yml  
|-- src/  
|-- examples/  
|-- tests/  
`-- output/
```


Paper-To-Spec Workflow

1. Extract

Identify equations, variables, inputs, outputs, and paper assumptions.

2. Name

Turn the method into modules, functions, data structures, and examples.

3. Validate

Test small analytical cases before reproducing paper-style results.

4. Explain

Document assumptions, approximations, and deviations from the paper.

Milestone Sequence

Contents of **MISSIONS.md**

1. Read **DATA.md** and write the implementation brief
2. Create package scaffold and environment files
3. Implement layered model extraction
4. Implement q-parameter equations
5. Implement direct two-point solver
6. Add travel time and ray path reconstruction
7. Add analytical and physical tests
8. Add plotting and one paper-style example
9. Add simple reflected rays if time permits
10. Document assumptions and limitations

Output Discipline

Contents of `OUTPUTS.md`

- source in `src/`
- examples in `examples/`
- tests in `tests/`
- generated images in `output/`
- documentation in `docs/`
- package metadata in `pyproject.toml` and `environment.yml`
- assumptions in `README.md`

Prompt 1: Start With a Plan Grounded with the Input Folder

Prompt 1

Analyze the files in input.
Perform the task described in TASK.md.
Use the paper context in DATA.md.
Prepare a step-by-step implementation plan that follows the milestones in MILESTONES.md.

Teaching point

The first prompt anchors the agent in the local task files and states the source-code boundary explicitly. The agent is asked to produce a plan that follows the milestones, which makes the milestones more likely to be followed and not forgotten.

Prompt 2: Convert Paper Context Into a Spec

Prompt 2

Before writing package code, extract the mathematical and software requirements from DATA.md.

Write a concise implementation brief that separates:

1. equations needed for the first pass,
2. assumptions we must make,
3. tests that can validate the implementation without relying on paper figures.

Teaching point

Ask for the model of the work before asking for the work itself.

This makes the agent expose uncertainty early.

Prompt 3: Implement the Solver Kernel

Prompt 3

Implement the q-parameter solver kernel from the paper:
q-to-p conversion, p-to-q conversion, offset $X(q)$, first derivative, second derivative, and an initial estimate.
Add numerical derivative checks and monotonicity tests for the offset equation.
Keep formulas documented in docstrings.

Teaching point

This prompt asks for implementation and validation together.
The test suite becomes the memory of the scientific method.

Prompt 4: Make Results Visible

Prompt 4

Add a plotting helper for 2-D ray paths over a layered velocity model.
Create `examples/01_basic_raytracing.py` that saves `output/laytracer_basic_01.png`.
The plot should show the layered velocity background, source, receivers, and ray paths clearly.

Prompt 5: Prepare Documentation and Reproducibility Files

Prompt 5

Prepare documentation and reproducibility files.
Create `README.md`, `pyproject.toml`, and `environment.yml` with installation, usage, and dependency information.
Generate Sphinx documentation with API references and usage examples in `docs/`.

Teaching point

Documentation and reproducibility files are often afterthoughts, but they should be part of the initial implementation plan to ensure the final product is usable and shareable.

Resulting Package: Structure and Content

Resulting implementation is available at <https://github.com/danikiev/LayTracer>

Readme

README

MIT license

LayTracer

DOI

Pytest passing

Docs passing

License MIT

Fast two-point seismic ray tracing in layered media.

LayTracer is an open-source Python package for computing ray paths, travel times, and amplitude attributes in horizontally layered (1D) velocity models with constant layer velocities. It is based on the dimensionless ray parameter method of Fang & Chen (2019), achieving rapid convergence.

Documentation: <https://danikiev.github.io/LayTracer>

Features

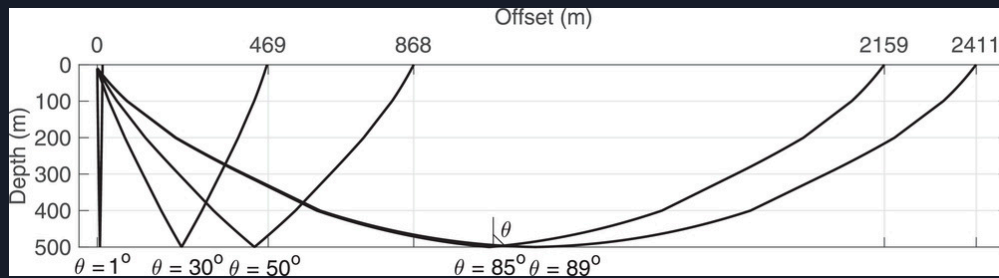
Category	Capability
Ray tracing	Second-order (quadratic) Newton solver using the dimensionless q -parameter for robust, singularity-free convergence
Travel time	Layer-by-layer travel time summation from the solved ray parameter
Attenuation	Intrinsic absorption operator t^* from quality factors Q
Spreading	Relative geometrical spreading from the analytical ray-tube Jacobian $\partial X/\partial p$
Reflection/Transmission	Full angle-dependent Zoeppritz P-SV coefficients (all 8 R/T modes) with optional energy-flux normalization (Červený, 2001)
Brewster angles	Automatic detection of Brewster-like zeros in R/T coefficient curves
Parallel execution	Multi-ray tracing with <code>joblib</code> / <code>loky</code> backend for large surveys
Visualisation	2-D ray path plots (matplotlib) and interactive 3-D viewer (Plotly)
Documentation	Comprehensive Sphinx docs with extensive theory, gallery examples, and API reference

Project structure

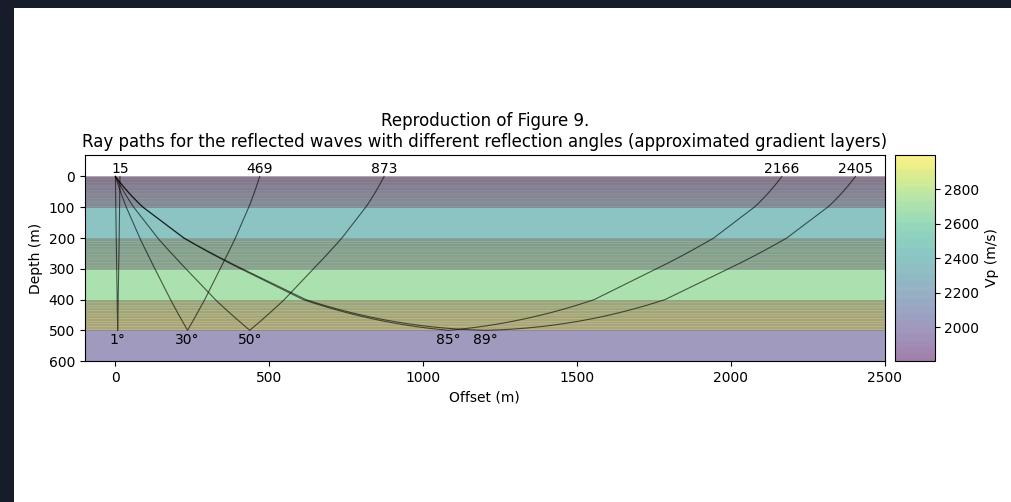
.github/workflows	Dev (#6)
docs	Dev (#6)
examples	Dev (#6)
laytracer	Dev (#6)
pytests	Dev (#6)
.gitignore	updates in documentation and the style of references
.zenodo.json	Dev (#6)
CHANGELOG.md	Dev (#6)
LICENSE	Initial commit
README.md	Dev (#6)
RELEASE.md	Merge pull request #4 from danikiev/dev
init.py	Initial commit
build-docs.bat	Added pytest.yml for github actions
build-docs.sh	updated git index for sh scripts
environment.yml	fixed numpy version and info about it
install.bat	Initial commit
install.sh	updated git index for sh scripts
pyproject.toml	fixed numpy version and info about it
pytest.ini	Clean up of dependencies and project toml

Resulting Package: Ability to Reproduce Paper Examples

Original Figure 9



Reproduced Figure 9

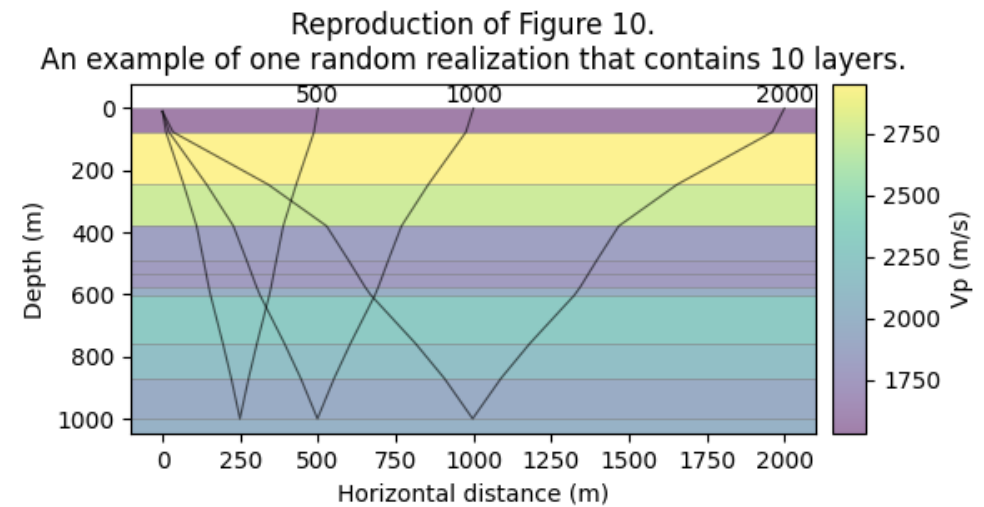


Resulting Package: Ability to Reproduce Paper Examples

Original Figure 10



Reproduced Figure 10

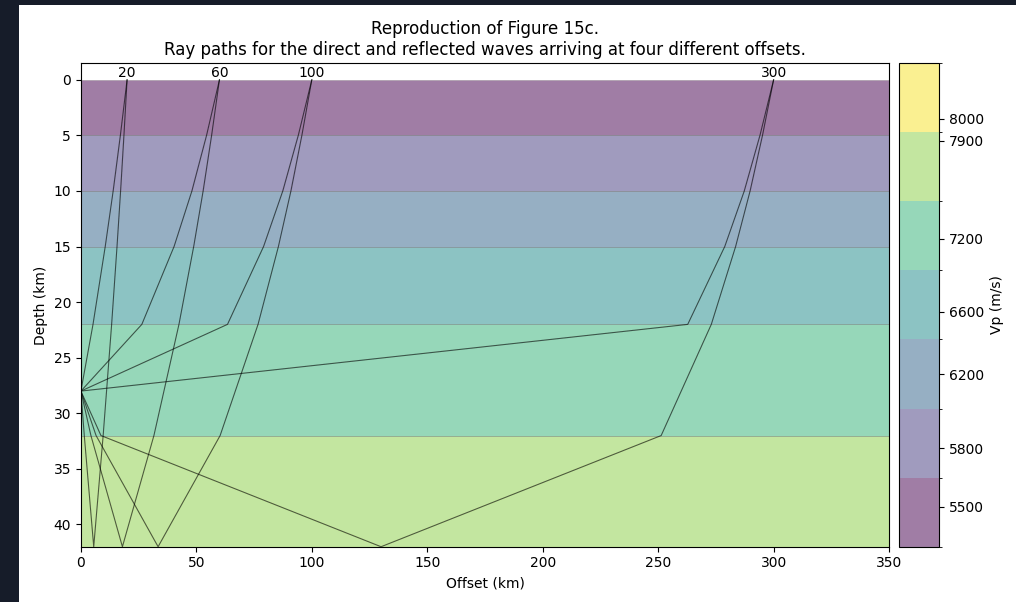


Resulting Package: Ability to Reproduce Paper Examples

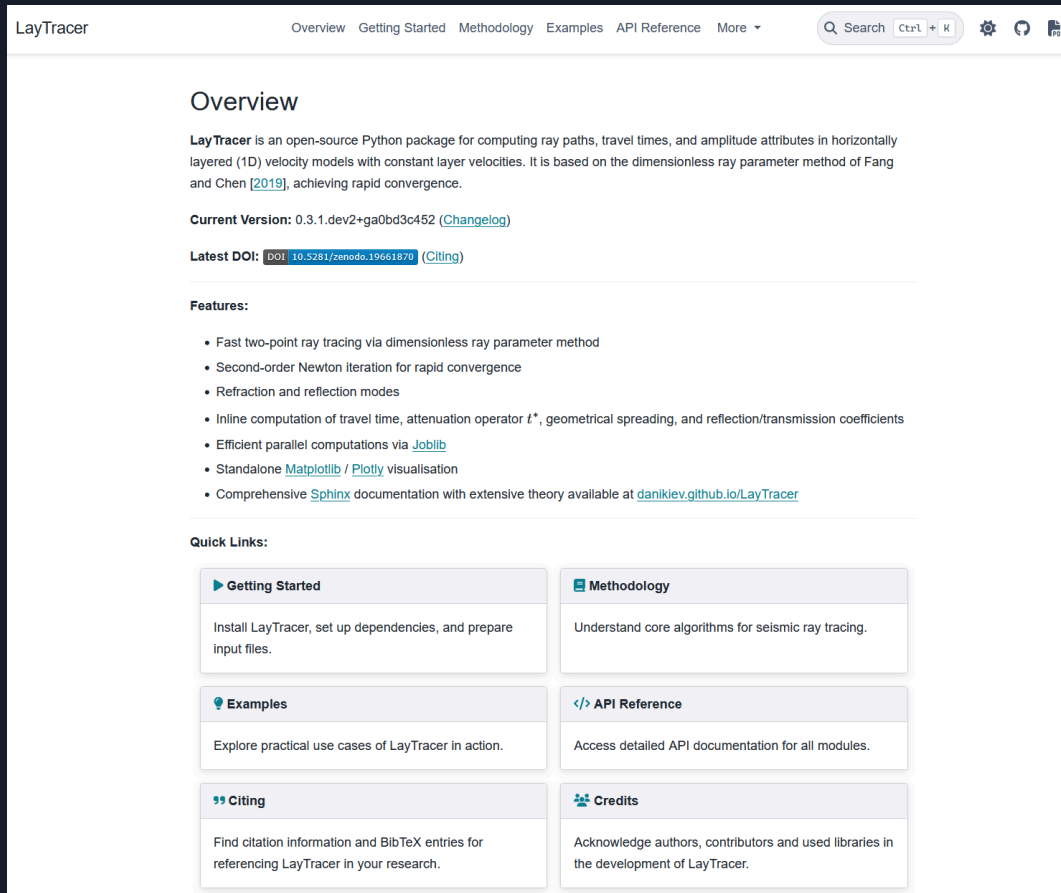
Original Figure 15c



Reproduced Figure 15c



Resulting Package: Website



The screenshot shows the LayTracer website documentation. The header includes the title 'LayTracer' and navigation links: Overview, Getting Started, Methodology, Examples, API Reference, and More. A search bar is also present. The main content area is titled 'Overview' and describes LayTracer as an open-source Python package for computing ray paths, travel times, and amplitude attributes. It mentions the current version (0.3.1.dev2+ga0bd3c452) and the latest DOI (10.5281/zenodo.19661870). Below this, a 'Features' section lists several capabilities: fast two-point ray tracing, second-order Newton iteration, refraction and reflection modes, inline computation of travel time and attenuation, efficient parallel computations via Joblib, standalone Matplotlib/Plotly visualisation, and comprehensive Sphinx documentation. A 'Quick Links' section at the bottom provides shortcuts to 'Getting Started', 'Methodology', 'Examples', 'API Reference', 'Citing', and 'Credits'.

LayTracer

Overview Getting Started Methodology Examples API Reference More

Search Ctrl+K

Overview

LayTracer is an open-source Python package for computing ray paths, travel times, and amplitude attributes in horizontally layered (1D) velocity models with constant layer velocities. It is based on the dimensionless ray parameter method of Fang and Chen [2019], achieving rapid convergence.

Current Version: 0.3.1.dev2+ga0bd3c452 (Changelog)

Latest DOI: DOI 10.5281/zenodo.19661870 (Citing)

Features:

- Fast two-point ray tracing via dimensionless ray parameter method
- Second-order Newton iteration for rapid convergence
- Refraction and reflection modes
- Inline computation of travel time, attenuation operator t^* , geometrical spreading, and reflection/transmission coefficients
- Efficient parallel computations via Joblib
- Standalone Matplotlib / Plotly visualisation
- Comprehensive Sphinx documentation with extensive theory available at danikiev.github.io/LayTracer/

Quick Links:

Getting Started Install LayTracer, set up dependencies, and prepare input files.	Methodology Understand core algorithms for seismic ray tracing.
Examples Explore practical use cases of LayTracer in action.	API Reference Access detailed API documentation for all modules.
Citing Find citation information and BibTeX entries for referencing LayTracer in your research.	Credits Acknowledge authors, contributors and used libraries in the development of LayTracer.

- The resulting package has a full documentation website with:
 - installation instructions
 - methodology
 - usage examples
 - API reference
 - Convenient navigation
 - PDF version of the documentation
- Check the website at <https://danikiev.github.io/LayTracer/>

Conclusions

- Demo 2 is about turning a paper into a tested package scaffold.
- The hard part is specifying the method, interfaces, and validation criteria.
- Focused prompts let the agent work like a careful junior collaborator.
- Tests are the strongest guardrail against plausible but wrong scientific code.
- The final product should be reproducible, inspectable, and honest about assumptions.